

usually involve **alliances**, whether formal or informal, among the players. Alliances are made and broken as the game proceeds. How are we to understand such behavior? Are alliances a natural consequence of optimal strategies for each player in a multiplayer game? It turns out that they can be.

Alliance

For example, suppose A and B are in weak positions and C is in a stronger position. Then it is often optimal for both A and B to attack C rather than each other, lest C destroy each of them individually. In this way, collaboration emerges from purely selfish behavior. Of course, as soon as C weakens under the joint onslaught, the alliance loses its value, and either A or B could violate the agreement.

In some cases, explicit alliances merely make concrete what would have happened anyway. In other cases, a social stigma attaches to breaking an alliance, so players must balance the immediate advantage of breaking an alliance against the long-term disadvantage of being perceived as untrustworthy. See [Section 18.2](#) for more on these complications.

If the game is not zero-sum, then collaboration can also occur with just two players. Suppose, for example, that there is a terminal state with utilities $\langle v_A = 1000, v_B = 1000 \rangle$ and that 1000 is the highest possible utility for each player. Then the optimal strategy is for both players to do everything possible to reach this state—that is, the players will automatically cooperate to achieve a mutually desirable goal.

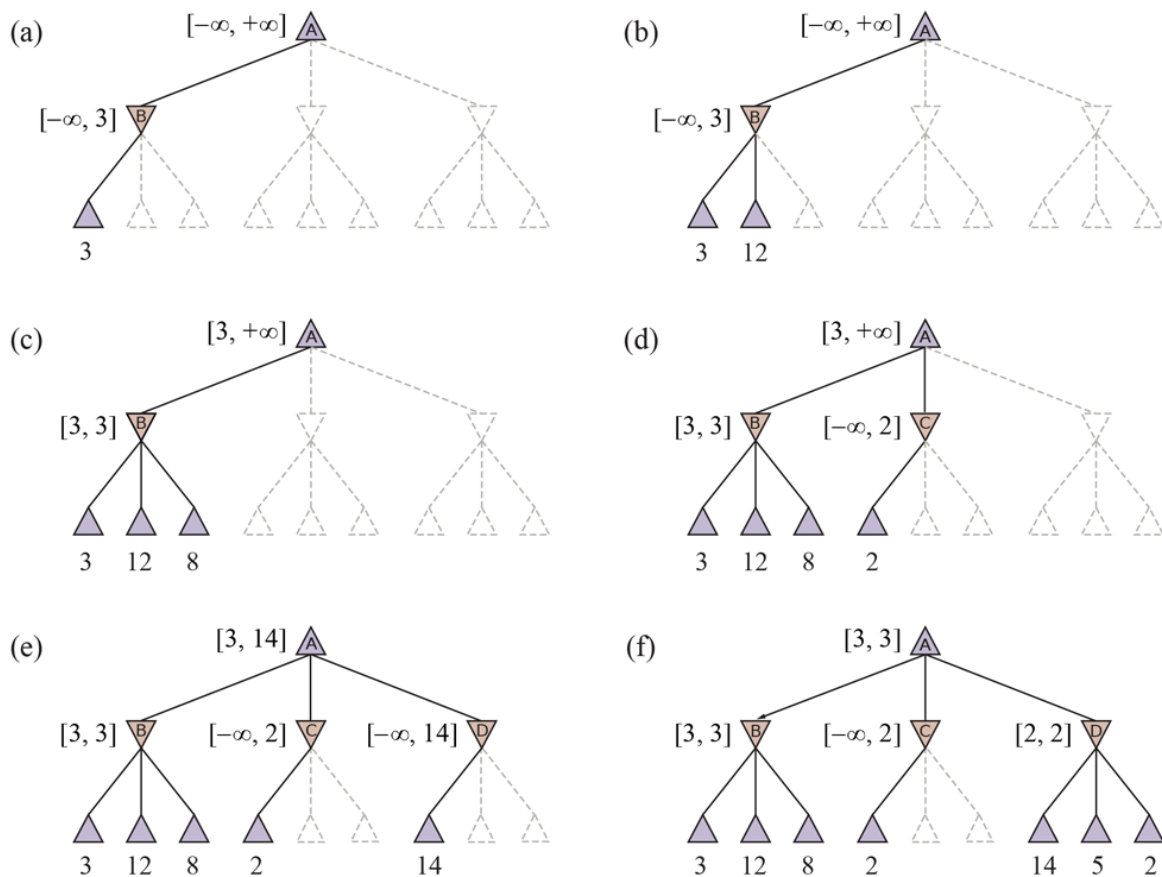
5.2.3 Alpha–Beta Pruning

The number of game states is exponential in the depth of the tree. No algorithm can completely eliminate the exponent, but we can sometimes cut it in half, computing the correct minimax decision without examining every state by **pruning** (see page 90) large parts of the tree that make no difference to the outcome. The particular technique we examine is called **alpha–beta pruning**.

Alpha-beta pruning

Consider again the two-ply game tree from Figure 5.2. Let's go through the calculation of the optimal decision once more, this time paying careful attention to what we know at each point in the process. The steps are explained in Figure 5.5. The outcome is that we can identify the minimax decision without ever evaluating two of the leaf nodes.

Figure 5.5



Stages in the calculation of the optimal decision for the game tree in Figure 5.2. At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B, which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still at most 3. (c) The third leaf below B has a value of 8; we have seen all B's successor states, so the value of B is exactly 3. Now we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C, which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C. Therefore, there is no point in looking at the other successor states of C. This is an example of alpha-beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also at most 14. (f) The second

successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B , giving a value of 3.

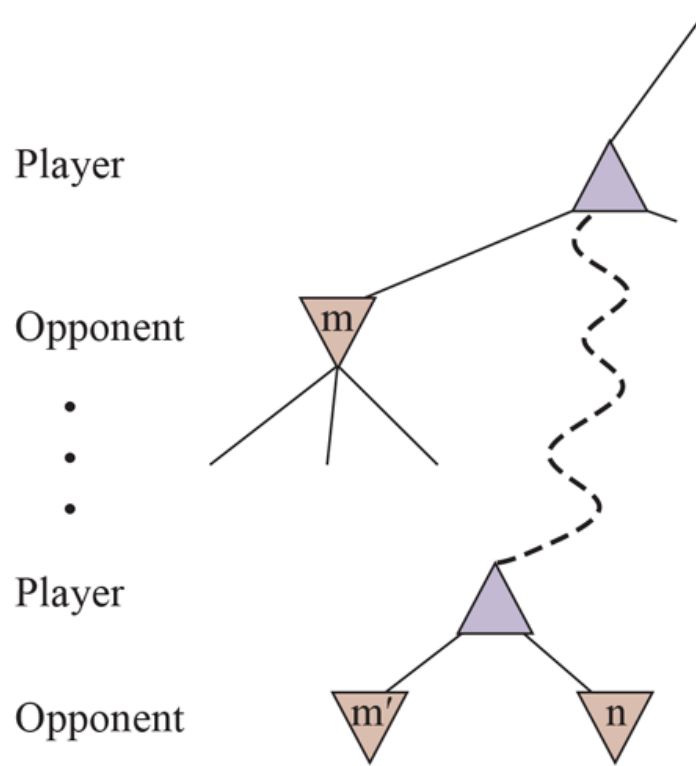
Another way to look at this is as a simplification of the formula for MINIMAX. Let the two unevaluated successors of node C in Figure 5.5 have values x and y . Then the value of the root node is given by

$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2)) \\ &= \max(3, \min(2,x,y), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2,x,y) \leq 2 \\ &= 3.\end{aligned}$$

In other words, the value of the root and hence the minimax decision are *independent* of the values of the leaves x and y , and therefore they can be pruned.

Alpha-beta pruning can be applied to trees of any depth, and it is often possible to prune entire subtrees rather than just leaves. The general principle is this: consider a node n somewhere in the tree (see Figure 5.6), such that Player has a choice of moving to n . If Player has a better choice either at the same level (e.g. m' in Figure 5.6) or at any point higher up in the tree (e.g. m in Figure 5.6), then Player will never move to n . So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.

Figure 5.6



The general case for alpha-beta pruning. If m or m' is better than n for Player, we will never get to n in play.

Remember that minimax search is depth-first, so at any one time we just have to consider the nodes along a single path in the tree. Alpha-beta pruning gets its name from the two extra parameters in MAX-VALUE ($state, \alpha, \beta$) (see [Figure 5.7](#)) that describe bounds on the backed-up values that appear anywhere along the path:

α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX. Think: α = "at least."

β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN. Think: β = "at most."

Figure 5.7

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action

player $\leftarrow$ *game*.TO-MOVE(*state*)

value, *move* $\leftarrow$ MAX-VALUE(*game*, *state*, $-\infty$, $+\infty$)

return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state*, *player*), *null*

v $\leftarrow -\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MIN-VALUE(*game*, *game*.RESULT(*state*, *a*), α , β)

if *v2* > *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

$\alpha \leftarrow$ MAX(α , *v*)

if *v* $\geq \beta$ **then return** *v*, *move*

return *v*, *move*

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state*, *player*), *null*

v $\leftarrow +\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MAX-VALUE(*game*, *game*.RESULT(*state*, *a*), α , β)

if *v2* < *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

$\beta \leftarrow$ MIN(β , *v*)

if *v* $\leq \alpha$ **then return** *v*, *move*

return *v*, *move*

The alpha-beta search algorithm. Notice that these functions are the same as the MINIMAX-SEARCH functions in Figure 5.3, except that we maintain bounds in the variables α and β , and use them to cut off search when a value is outside the bounds.

Alpha-beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively. The complete algorithm is given in Figure 5.7. Figure 5.5 traces the progress of the algorithm on a game tree.

5.2.4 Move ordering

The effectiveness of alpha-beta pruning is highly dependent on the order in which the states are examined. For example, in Figure 5.5(e) and (f), we could not prune any

successors of D at all because the worst successors (from the point of view of MIN) were generated first. If the third successor of D had been generated first, with value 2, we would have been able to prune the other two successors. This suggests that it might be worthwhile to try to first examine the successors that are likely to be best.

If this could be done perfectly, alpha–beta would need to examine only $O(b^{m/2})$ nodes to pick the best move, instead of $O(b^m)$ for minimax. This means that the effective branching factor becomes $\sqrt{b}$ instead of b —for chess, about 6 instead of 35. Put another way, alpha–beta with perfect move ordering can solve a tree roughly twice as deep as minimax in the same amount of time. With random move ordering, the total number of nodes examined will be roughly $O(b^{3m/4})$ for moderate b . Now, obviously we cannot achieve *perfect* move ordering—in that case the ordering function could be used to play a perfect game! But we can often get fairly close. For chess, a fairly simple ordering function (such as trying captures first, then threats, then forward moves, and then backward moves) gets you to within about a factor of 2 of the best-case $O(b^{m/2})$ result.

Adding dynamic move-ordering schemes, such as trying first the moves that were found to be best in the past, brings us quite close to the theoretical limit. The past could be the previous move—often the same threats remain—or it could come from previous exploration of the current move through a process of **iterative deepening** (see page 80). First, search one ply deep and record the ranking of moves based on their evaluations. Then search one ply deeper, using the previous ranking to inform move ordering; and so on. The increased search time from iterative deepening can be more than made up from better move ordering. The best moves are known as **killer moves**, and to try them first is called the killer move heuristic.

Killer moves

In Section 3.3.3 , we noted that redundant paths to repeated states can cause an exponential increase in search cost, and that keeping a table of previously reached states can address this problem. In game tree search, repeated states can occur because of **transpositions**—different permutations of the move sequence that end up in the same